WASHWISE: A Smart Laundry Management System with YOLO-Based Clothing Detection

MOHAMADFAYAS N
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
mohamadfayas999@gmail.com

SIVARAM G
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
sivaram.svgk@gmail.com

Dr. P. Revathy
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India

***Abstract*—Shared washing machines in hostels are often booked through informal coordination, and counting garments alone does not show whether a load fits a machine or a fair-use limit. This paper presents WASHWISE, a smart laundry management system developed through a Design Thinking process for two separate contexts: Rajalakshmi Engineering College (REC) hostels, where a staff-mediated workflow covers intake, storage and complaints, and a paying-guest (PG) demonstration environment with resident self-service. Users enter garment counts manually or use a camera-assisted workflow in which a separate Flask service runs an open-source, Fashionpedia-trained YOLOv8 Nano model. Detected categories are mapped to five laundry classes and converted into an estimated weight using configured category values. The estimate is validated against a 4,500 g session limit and a 7,000 g weekly allowance that resets every Monday at 12:00 AM, with automatic multi-session splitting and rejection of any booking that cannot be reserved in full. The system uses React, Express, Prisma and PostgreSQL. The repository contains 18 backend integration test suites and one Python AI test suite; their coverage is described, but recorded pass results, detection accuracy, latency and usability measurements are not available and are left as future work. Weight is estimated from detected categories, not measured by the camera.**

***Index Terms*—laundry management, YOLOv8, clothing detection, weight estimation, Design Thinking, role-based access control**

I. Introduction

Residents of college hostels and paying-guest (PG) accommodation share a small number of washing machines. When booking depends on informal coordination, residents cannot easily see which slots are free, and a user's load is judged by the number of garments rather than by what the machine can actually take. Item counts are a poor proxy for load: a bedsheet, a pair of pants and a T-shirt differ greatly in mass. Large loads also have to be divided by hand, and per-person usage limits are difficult to track.

WASHWISE is a web application that addresses these problems in two operating contexts. In the first, Rajalakshmi Engineering College (REC), students of three hostels (Habitat, Thandalam and Girls Main) use an institutionally managed service in which laundry staff and a college administrator take part. In the second, a separate PG environment lets residents manage their own wash sessions without laundry staff. In both, a user supplies garment information either by manual entry or by a photograph analysed by a YOLO-based detection service. The system turns the result into an estimated load weight, validates it against session and weekly limits, and reserves one or more slots.

The objectives are to (i) provide digital slot booking for both contexts, (ii) reduce manual counting through camera-assisted detection, (iii) estimate load weight from garment categories, (iv) enforce a 4,500 g session limit and a 7,000 g weekly allowance, and (v) keep REC and PG users and administrators strictly separate.

The paper is a practical system design and implementation report. Its contributions are:

1. an integrated laundry system supporting both an institutionally managed REC workflow and a PG self-service workflow;
2. integration of YOLO-based clothing detection into the booking flow through a separate inference service;
3. category-based weight estimation connected to session and weekly allowance rules, including automatic multi-session booking;
4. digital booking, order tracking and the staff and administrative workflows needed for the REC context; and
5. a modular architecture that separates the frontend, backend, database and AI inference service.

No claim of novelty beyond this combination is made, and no performance advantage is claimed because none has been measured. The remainder of the paper is organised as follows. Section II surveys related work, and Section III explains how the Design Thinking process was applied. The architecture and implementation are described in Sections IV and V, Section VI sets out the test evidence that is available, and Sections VII to IX cover the discussion of results, the limitations and future work, and the conclusion.

II. Related Work

A. Digital Laundry Management

Serajudin and Mostafa describe a self-service booking application for a residential college that replaces a first-come, first-served laundry arrangement with slot reservation, machine-availability checking and notifications on booking and cycle completion [1]. Portase et al. propose SmartLaundry, which combines machine status from sensors with usage

forecasts to recommend machines in public laundries; they validate the recommendation logic by simulation rather than deployment [2]. Both works show the value of visible availability and booking. They do not, as far as their abstracts indicate, estimate load weight from garment images or apply per-user weekly weight allowances, which is the combination WASHWISE explores. WASHWISE does not use machine sensors or usage forecasting.

B. Object Detection and YOLO

YOLO uses one network that examines the whole image a single time and directly regresses bounding boxes together with class probabilities, so detection needs only one forward pass [3]. The development of the YOLO family from YOLOv1 up to YOLOv8 and YOLO-NAS is reviewed in a survey by Terven et al. [4], and YOLOv8 is made available through the Ultralytics library [5]. WASHWISE uses the Nano variant, which its AI service runs on CPU.

C. Clothing Recognition

DeepFashion provides large-scale clothing images with rich annotations for recognition and retrieval [6]. Fashionpedia defines an ontology of 27 main apparel categories together with garment parts and attributes, with segmentation masks and localisation [7]. A detector trained on this ontology can therefore name garments (and also garment parts such as sleeves or collars). Both datasets consist mostly of catalogue-style and worn-garment images, while laundry photographs often contain folded or overlapping garments. Detector behaviour on such images therefore cannot be taken for granted.

D. Image-Assisted Load Estimation

An ordinary photograph does not reveal mass. A practical estimate therefore has to combine recognised categories with assumed per-category values. This is the approach taken here, and its limits (fabric, size, thickness) are discussed in Section VIII.

III. Design Thinking Methodology

The project followed the five stages of Design Thinking [8]. The description below is limited to activities supported by the project documents; no interview counts, survey figures or user-study results are reported, because none were recorded.

Empathize. Four stakeholder groups were considered from the authors' understanding of hostel life: REC students, PG residents, laundry staff and administrators. The stage was qualitative. It surfaced uncertainty about slot availability, difficulty judging whether a load fits a machine, and (in the REC context) uncertainty about garment counts and storage after hand-over.

Define. The problem was stated as the need for a booking system that accounts for load weight rather than item count, limits usage fairly, and handles loads larger than one machine session, while keeping college and PG users separate.

Ideate. Ideas included slot booking, manual and camera-assisted garment entry, category-based weight estimation, a per-session limit, a weekly allowance, automatic splitting of large loads, recommended slots, and, for REC, staff verification, rack/shelf storage and a complaint workflow.

Prototype. The ideas were built as a React web application with a Node.js/Express backend, a PostgreSQL database and a separate Flask detection service. A Capacitor wrapper provides native camera access on mobile devices.

Test. Verification so far consists of the automated test suites described in Section VI. A usability study with real users has not been conducted and is future work.

Table I
Main Technologies

Component	Technology (versions from project manifests)
Frontend	React 18.3.1, TypeScript 5.6.3, Vite 5.4.11, Tailwind CSS 3.4.15, React Router 6.28, Framer Motion; Capacitor camera plugin
Backend	Node.js, Express 4.21.1, TypeScript, JWT, bcryptjs
Data	PostgreSQL 16, Prisma ORM 5.22.0 (14 models)
AI service	Python 3.10+, Flask, Gunicorn, Ultralytics YOLO, PyTorch, Pillow, OpenCV
Hosting	Frontend on Vercel; backend and database on Render; AI service deployed separately

IV. System Architecture and Workflow

A. Architecture

WASHWISE uses four loosely coupled components (Fig. 1). The frontend is a React single-page application that provides views tailored to each user role and takes care of capturing garment photographs. The *backend* is an Express REST API that authenticates users, enforces role, environment and hostel rules, applies the booking logic and mediates all database access through Prisma. A PostgreSQL *database* stores users, hostels, slots, bookings, orders, notifications, complaints and audit records. A separate Python *AI service* performs image-based clothing detection; the backend forwards images to it over HTTP and never exposes it directly to the browser. Table I lists the main technologies.

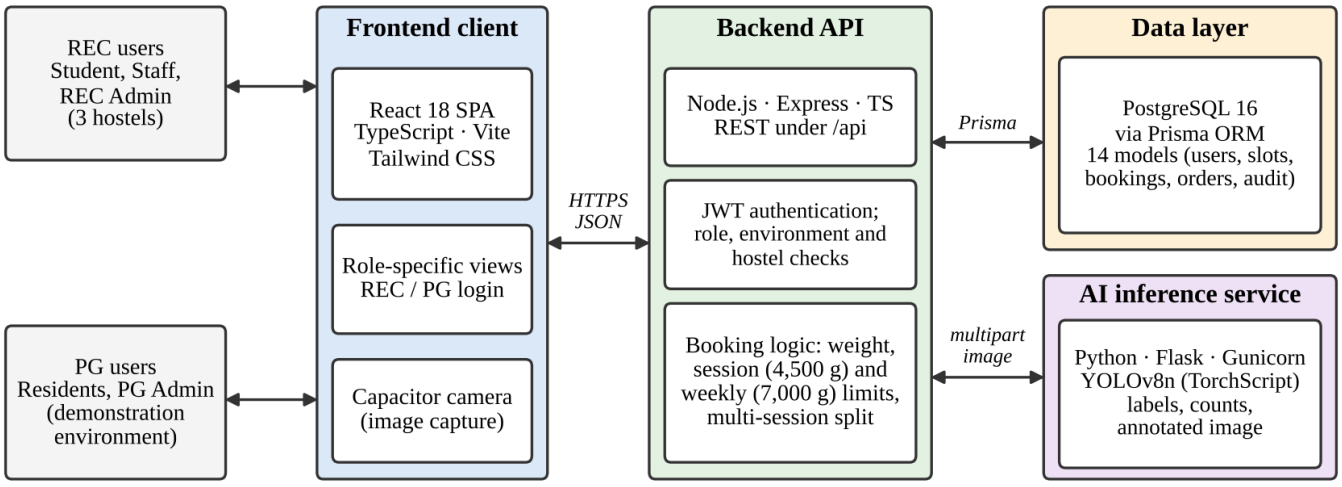


Fig. 1. WASHWISE system architecture. REC and PG users reach the same React client; the Express backend applies access and booking rules, stores data in PostgreSQL through Prisma and calls the separate Flask/YOLO inference service when a photograph is submitted.

B. Environments, Roles and Authentication

The platform has three roles (student or resident, staff and administrator) and two environments, `COLLEGE` (REC) and `PG_SHARED`. A login request carries a facility identifier. The server checks the role, the password hash and the facility, then issues a JSON Web Token (JWT) [10] that carries an environment claim; middleware rejects a token whose environment disagrees with the database record. Passwords are stored as bcrypt hashes [11]. Authorisation follows role-based access control [9], extended with hostel scope: staff and students reach only their own hostel's data, a REC administrator cannot see PG data, and a PG administrator cannot see REC data. Staff accounts exist only in the REC environment.

The PG environment in the current demonstration consists of four individual user accounts and a separate PG administrator account. These are demonstration accounts; they are not four independently deployed or validated PG facilities.

C. Booking, Estimation and Allowance Rules

Let n_c be the number of garments of category c and w_c a configured per-category weight. The estimated load is

$$W = \sum_c n_c w_c. \quad (1)$$

The rules applied to every booking in both environments are as follows.

- A single session carries at most $L_s = 4{,}500$ g.
- Each user has a weekly allowance of $A = 7{,}000$ g, which resets every Monday at 12:00 AM.
- The allowance is deducted when the booking is made, and an eligible cancellation before the slot starts restores it.
- If $W > L_s$, the load can be split into $k = \lceil W/L_s \rceil$ sessions (for example, 6,000 g becomes 4,500 g plus 1,500 g), provided enough free slots and enough remaining allowance exist.
- If the complete multi-session booking cannot be reserved, the whole request is rejected; no partial booking is created.

A booking is therefore not guaranteed: it succeeds only when a suitable slot and sufficient allowance are both available. A washing cycle is intended to take about one hour.

D. Order Lifecycle and the Two Workflows

Table II and Fig. 2 contrast the two contexts. In REC, a student books slots and declares garments; staff physically verify and lock the count, assign a free rack/shelf location (three racks of four shelves, 12 locations per hostel depot), process the load and mark it complete, which creates an in-app notification. The student then verifies receipt or files a complaint (torn clothes, reduced count or other issue), which the REC administrator resolves; resolution releases the storage location. Rack and shelf identifiers and staff identities are removed from student-facing responses. In the PG environment no staff are involved: the resident starts and finishes the wash through dedicated endpoints, moving the order from booked to in progress to completed.

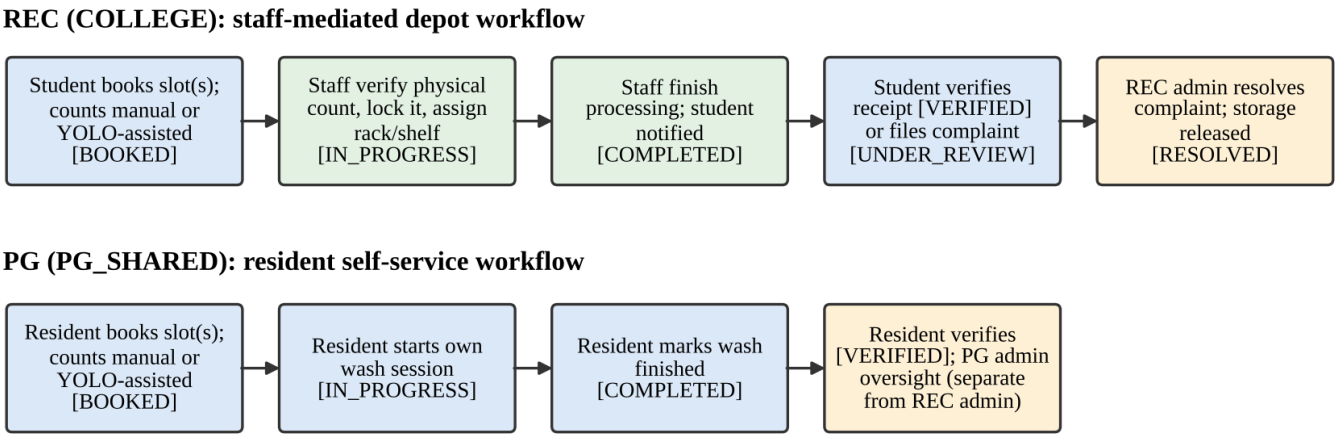


Fig. 2. Order lifecycle in the two contexts. Upper row: staff-mediated REC workflow. Lower row: PG resident self-service workflow, with its own administrator separate from the REC administrator.

Table II
REC AND PG WORKFLOW COMPARISON

Aspect	REC (`COLLEGE`)	PG (`PG_SHARED`)
Facilities	Habitat, Thandalam, Girls Main	Four demonstration user accounts
Roles	Student, staff, REC admin	Resident, PG admin (no staff)
Wash cycle	Staff intake, processing, completion	Resident starts and finishes
Storage	Rack/shelf assigned by staff	Not applicable
Administrator	REC admin (three hostels)	Separate PG admin
Shared rules	4,500 g session; 7,000 g weekly; multi-session split	

V. IMPLEMENTATION

A. Booking and Slot Management

Slots are hourly and capacity-capped per hostel. The booking endpoints validate slot availability, the session limit and the remaining weekly allowance in one transaction. A planning endpoint returns an advisory split for a given load, and a multi-session endpoint creates linked reservations under one batch identifier so that either all sessions are reserved or none is. Cancellation is allowed only while a booking is still in the booked state.

B. Garment Entry and YOLO-Assisted Detection

Users may enter category counts manually or take a photograph (Fig. 3). The backend accepts JPEG, PNG or WebP files up to 10 MB and forwards them to the Flask service, which runs the detector at a 640-pixel input size with a default confidence threshold of 0.45, configurable by environment settings. The model is an open-source YOLOv8 Nano network trained on the 46-class Fashionpedia ontology and distributed as a 12.5 MB TorchScript file; it is a pre-trained checkpoint obtained from a public repository (`louisJLN/yolo8-fashionpedia`, as recorded in the project audit), and the project records show no training or fine-tuning by the authors. Inference runs on CPU with a single worker.

The service maps Fashionpedia classes to operational categories: “shirt, blouse” to *shirt*; “top, t-shirt, sweatshirt” to *tshirt*; *pants*; *shorts*; and other wearable apparel (for example sweaters, jackets, skirts, coats and dresses) to *other*. Garment parts (classes 27–45, such as collars, sleeves and pockets) and accessories are excluded so that one garment is not counted several times. The response contains the detections, per-category counts, average confidence and an annotated image encoded in Base64. Uploaded images are written to temporary files and deleted in a `finally` block after inference, and only derived counts and categorical values are stored in the database.

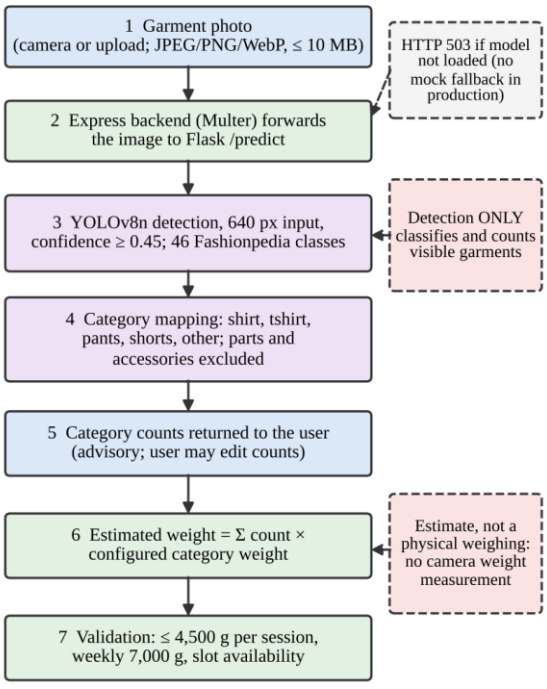


Fig. 3. YOLO-assisted clothing estimation pipeline. The detector only classifies and counts visible garments; weight is estimated from category counts and configured values, not measured from the image.

Detection output is advisory: the user can edit the counts before booking, and REC staff verify the physical count at intake. In production mode, if the model cannot be loaded the service returns HTTP 503 rather than falling back to simulated detections; a mock detector exists only for explicit development mode.

C. Weight Estimation and Allowance Management

The confirmed category counts are combined with the configured weights as in Eq. (1). The result feeds the booking validation described in Section IV-C. Because the weights are fixed values per category, the estimate is deterministic and auditable, and it can be corrected by changing configuration rather than retraining the model.

D. REC Staff, Storage and Administration

REC staff see only their own hostel's bookings, look up students by roll number or an opaque digital identifier (`ww_qr_` plus random hexadecimal characters, displayed on the student profile; staff do not scan it with a camera), confirm and lock the count, and allocate a free rack/shelf location, which is enforced as unique per hostel. Marking an order complete insert a notification for the student. The REC administrator sees summary counts, demand insights (peak slots, busiest depot and a rolling seven-day trend computed from stored rows), a complaint queue, a storage monitor, student and staff directories, and an audit log recording actor, action, entity and before/after values. The PG administrator has a separate view over the PG environment.

E. Order Tracking and Complaints

Each order moves through explicit states: booked, in progress, completed, verified, or under review and then resolved if a complaint is filed. A history table keeps a denormalised record of each finished order. Together these let a user see the status of a booking, and let an administrator trace a dispute to the recorded count and storage location.

VI. Testing and Evaluation

The evidence available for this paper is the result of a read-only technical audit of the repository, which identified 18 backend integration test suites and one dedicated Python AI test suite.

Table III
Areas Covered by Existing Test Suites

Area	Intended coverage (from suite descriptions)
Foundation, API	Database connectivity, seeded data, password hashing, health and routing
Authentication	Login for REC and PG accounts, rejection of invalid credentials, staff blocked from PG facilities
Isolation	Cross-facility and cross-tenant boundaries
Booking	Slot reservation limits, cancellation edge cases, weight calculation, multi-batch bookings
Staff and storage	Intake, count confirmation, rack/shelf allocation
Lifecycle	Completion, notification, verification, complaint, resolution and storage release, status transitions
Admin	Dashboard analytics, complaint resolution, audit logs, cross-hostel filters
AI pre-scan	Recommendation rules, access guards, scan-to-booking linkage for residents

The audit lists what these suites are designed to exercise; it does not include recorded execution logs. The suites are therefore reported as areas of test coverage, and *this paper does not state that every test passed*.

The Python suite calls the Flask application through its test client. Its nine checks cover model initialisation and the 46-class mapping; the health endpoint; detection on a real T-shirt photograph; a non-garment document image (an academic marksheet) that is expected to yield no garments; multi-garment images; a dark frame; a heavily blurred image; rejection of an unsupported `.txt` upload with HTTP 415; and the missing-model case, which must return HTTP 503 without mock fallback. These checks verify functional behaviour and robustness on only a few sample images, so they do not amount to a formal benchmark.

No precision, recall, F1-score or mAP values on laundry images, no inference latency or concurrency measurements, no success rates, no time-saving figures and no usability scores are available. The current evidence therefore supports an implementation-oriented, functional description; formal quantitative evaluation remains future work.

VII. Results and Discussion

Table IV separates what is implemented from what has been tested and measured. The system integrates its parts so that one booking request exercises the whole chain: a photograph or manual counts produce category counts, the counts produce an estimated weight, the weight is checked against the 4,500 g and 7,000 g limits and slot availability, and a successful booking deducts the allowance and creates one or more linked reservations. In the REC context the same record then passes through staff verification and storage; in the PG context it passes through the resident's own wash session.

Table IV
Evidence Status by Function

Function	Implemented	Suite exists	Result
REC/PG separation	Yes	Yes	Not recorded
Slot booking, split	Yes	Yes	Not recorded
YOLO detection	Yes	Yes	Not recorded
Weight estimate	Yes	Partly	Not recorded
Staff, storage	Yes	Yes	Not recorded
Complaints, audit	Yes	Yes	Not recorded
Accuracy, latency	—	—	Not measured
Usability	—	—	Not measured

Several design properties follow directly from the implementation rather than from measurement. Weight-based limits make the estimated load, rather than the item count, the unit of fair use. Rejecting incomplete multi-session bookings avoids the situation in which a user holds a single slot for a load that needs two. The separate AI service lets the detector be

replaced or retrained without changing the frontend or booking rules, and the explicit 503 behaviour keeps simulated output from being mistaken for real inference. Image privacy is handled by deleting uploads immediately after inference. None of these properties has been shown to reduce waiting time, disputes or staff effort; such claims would require a field study.

VIII. Limitations and Future Work

A. Limitations

- *Estimated, not measured, weight.* Weight is a sum of category counts and configured values. Fabric, garment size and thickness cause real garments to differ, and the *other* category is especially coarse.
- *Image conditions.* Detection depends on lighting, image quality, garment visibility and overlap. Fashionpedia is not laundry-specific, so behaviour on folded or stacked clothes is unquantified; garments may be missed or merged.
- *No quantitative benchmark.* No accuracy, latency or concurrency figures exist; the CPU service with a single worker should not be assumed to be real-time under load.
- *Limited real-world validation.* The PG environment is a demonstration with four accounts and one administrator, and no usability study has been run. Security rests on standard practices (bcrypt, JWT, role and environment checks); no penetration test was performed.
- *Dependencies.* Use requires network access to the deployed services.

B. Future Work

Planned improvements, none of which is implemented, are: (i) evaluating the detector on a labelled set of real laundry photographs and fine-tuning if needed; (ii) calibrating category weights against measured garments; (iii) benchmarking the inference endpoint under concurrent load; (iv) a usability study with REC and PG users; (v) reminders and push notifications; and (vi) integration with smart machines for live availability.

IX. Conclusion

WASHWISE has been implemented as a completed, working laundry management system for two settings: a staff-mediated workflow for three REC hostels and a self-service demonstration environment for PG residents. The system brings together slot booking, manual or YOLO-assisted garment entry, category-based weight estimation, a 4,500 g session limit, and a 7,000 g weekly allowance with automatic multi-session splitting, along with access control scoped by role, environment and hostel. These parts sit in a modular architecture built on React, Express, PostgreSQL and Flask. Garment detection relies on a pre-trained, open-source Fashionpedia YOLOv8 Nano model, and what it yields is an estimate based on garment categories, not a physical weighing. The contribution lies in the practical integration of these components. Accuracy, speed and usability have yet to be measured; the natural next steps are an evaluation on labelled laundry images, load benchmarking and a user study.

Acknowledgment

The authors thank their guide, Dr. P. Revathy, the Head of the Department, Dr. S. Vinodh Kumar, the project coordinator, Dr. S. P. Karuppiah, and the Department of Computer Science and Design, Rajalakshmi Engineering College, for their guidance and support.

References

[1] N. D. S. Serajudin and S. A. Mostafa, "Self-service laundry booking application for Tun Dr Ismail Residential College," *Appl. Inf. Technol. Comput. Sci.*, vol. 6, no. 1, pp. 823–841, 2025.

[2] R. L. Portase, R. Tolas, and R. Potolea, "SmartLaundry: A real-time system for public laundry allocation in smart cities," *Sensors*, vol. 24, no. 7, Art. no. 2159, 2024, doi: 10.3390/s24072159.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[4] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Mach. Learn. Knowl. Extr.*, vol. 5, no. 4, pp. 1680–1716, 2023.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[6] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, "DeepFashion: Powering robust clothes recognition and retrieval with rich annotations," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 1096–1104.

[7] M. Jia *et al.*, "Fashionpedia: Ontology, segmentation, and an attribute localization dataset," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science. Springer, 2020, pp. 316–332, doi: 10.1007/978-3-030-58452-8_19.

[8] T. Brown, "Design thinking," *Harvard Bus. Rev.*, vol. 86, no. 6, pp. 84–92, Jun. 2008.

[9] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.

[11] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, 1999, pp. 81–91.